Assignment 7 - LoRA Fine-Tuning of LLaMA-2

1. Introduction

This project fine-tunes Meta's LLaMA-2-7B model using Low-Rank Adaptation (LoRA) on the Dolly-15K dataset to improve instruction-following performance. We then evaluate the fine-tuned model against the base model using AlpacaEval and MT-Bench (single-turn) benchmarks with GPT-4o-mini as an automatic evaluator.

2. Preparing Dolly-15k Dataset

The dataset used for this project was the **Databricks Dolly-15k** dataset from the Hugging Face Hub. It contains approximately **15,000** human-generated instruction–response pairs across a wide range of domains, including open-ended questions, brainstorming, and classification tasks. Each sample in the dataset consists of three fields: instruction, context, and response.

To prepare the data for instruction fine-tuning, several preprocessing steps were applied. First, all entries with empty or whitespace-only responses were removed to ensure that every sample had a valid target output. Next, each example was reformatted into a consistent **Alpaca-style format** to align with the expected prompt-response structure for fine-tuning LLaMA-based models.

If either the instruction or context fields were missing, they were replaced with "N/A" to maintain structural consistency. The dataset was then **shuffled using a fixed random seed (42)** to ensure reproducibility and balanced distribution before splitting. The cleaned dataset was divided into **80% training**, **10% validation**, and **10% test** subsets to support model training, hyperparameter tuning, and final evaluation respectively.

Finally, the processed dataset was saved to disk in **Hugging Face Arrow format** under the directory data/dolly/ for efficient loading during fine-tuning.

This preprocessing ensured that the data followed a uniform instruction–response structure, removed noisy or incomplete examples, and improved the model's ability to learn coherent, instruction-following behaviour during fine-tuning.

	Before (raw sample)	**After (formatted for fine-tuning)**
Instruction	"Explain how photosynthesis works."	"Explain how photosynthesis works."
Context	""	"N/A"
Response	"Photosynthesis is the process by which green plants use sunlight to convert water and carbon dioxide	"Photosynthesis is the process by which green plants use sunlight to convert water and carbon dioxide

	into glucose and oxygen."	into glucose and oxygen."

Table 1. Example of the formatting

3. Model Training

The base model used for fine-tuning was **Meta's LLaMA-2-7B (meta-llama/Llama-2-7b-hf)**. To enable efficient training on limited hardware, **Parameter-Efficient Fine-Tuning (PEFT)** was applied using the **Low-Rank Adaptation (LoRA)** technique. This approach modifies only a small subset of model parameters, significantly reducing GPU memory requirements while preserving model quality.

The fine-tuning was performed on the **Databricks Dolly-15k** dataset (processed as described earlier) to teach the model instruction-following behaviour. Training was carried out on **Google Colab Pro**, using an **NVIDIA A100 (High-RAM)** GPU over approximately **3 hours**.

LoRA Configuration

Parameter	Value	Description
r	16	LoRA rank – number of trainable low-rank dimensions
lora_alpha	32	Scaling factor applied to LoRA updates
target_modules	["q_proj", "k_proj", "v_proj", "o_proj"]	Layers modified by LoRA adapters
lora_dropout	0.05	Dropout applied to LoRA layers
bias	none	No additional bias parameters used
task_type	CAUSAL_LM	Causal language modeling objective

Table 2. LoRA Configuration

This configuration enables efficient gradient updates while maintaining model stability and avoiding overfitting

Training Configuration

Setting	Value
Optimizer	AdamW
Learning rate	2e-4
Epochs	3

Effective Batch size	16
Weight decay	0.01
Warmup ratio	0.03
Scheduler	Cosine
Max sequence length	512 tokens
Mixed precision	bfloat16
Gradient checkpointing	Enabled
Quantization	8-bit via bitsandbytes

Table 3. Training Configuration

Training Results

Training and validation loss were tracked over time. The model demonstrated **stable convergence**, with training loss decreasing steadily and validation loss flattening as expected for instruction-tuning tasks.
The final loss values stabilised around **1.11 (train)** and **1.32(validation)**, indicating a good balance between fit and generalisation.

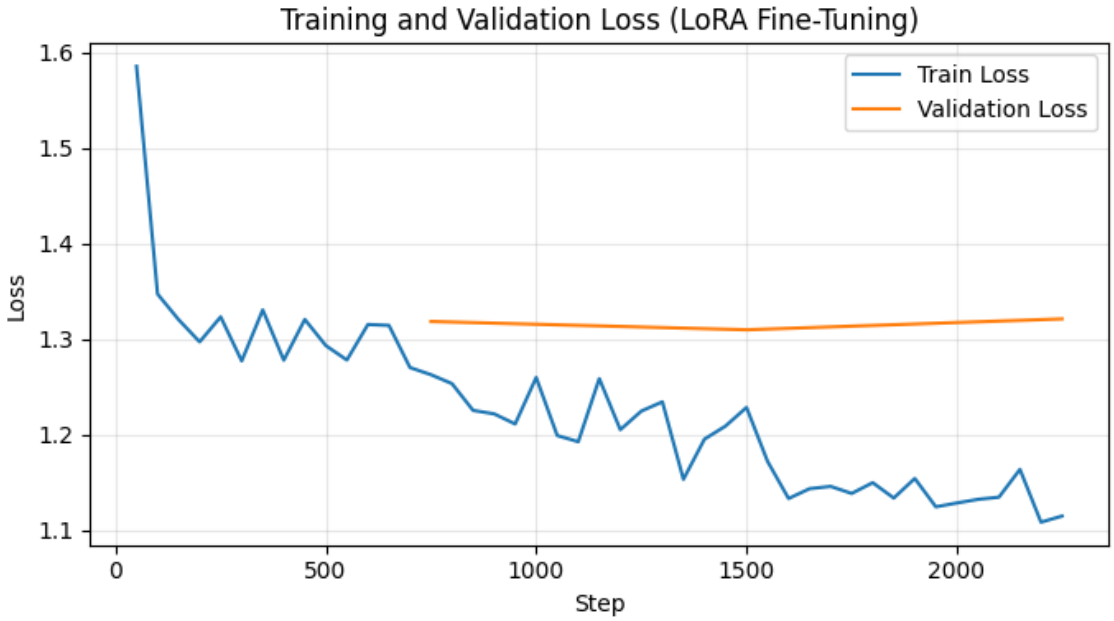


Image 1. Loss Curve

The fine-tuned model was saved as llama2_7b_dolly_lora_model, including both adapter weights and tokenizer files for reproducible inference.

4. Evaluation Methods

4.1 AlpacaEval

To evaluate the instruction-following capability of the fine-tuned model, the **AlpacaEval** benchmark was used to compare the **base LLaMA-2-7B** model with the **LoRA fine-tuned** version trained on the **Dolly-15k** dataset.
The experiment was conducted on **Google Colab Pro** using an **NVIDIA A100 High-RAM** GPU.

Evaluation Setup

Component	Description
Dataset	AlpacaEval (instruction–response dataset from Hugging Face)
Access Method	The dataset was downloaded and saved locally (alpaca_eval.json) because the GitHub URL repeatedly failed to load, and the Hugging Face web JSON preview cannot be directly parsed in Colab.
Base Model	meta-llama/Llama-2-7b-hf
Fine-Tuned Model	LoRA-adapted LLaMA-2-7B trained on Dolly-15k
Judge Model	GPT-4o-mini
Evaluation Samples	100
Metrics	Raw Win Rate and Length-Controlled Win Rate

Table 4. AlpacaEval Setup

Evaluation Method

1. **Response Generation** – Both models generated answers to identical instruction prompts.
2. **Pairwise Judging** – **GPT-4o-mini** compared the two responses and selected "A" (fine-tuned), "B" (base), or "Tie" based on correctness, helpfulness, and coherence.
3. **Metric Computation** –
    - **Raw Win Rate:** % of cases where the fine-tuned model was preferred.
    - **Length-Controlled Win Rate:** Adjusted score that removes response-length bias using logistic regression.

Results

Metric	Score	Description
Raw Win Rate	**78 %**	Fine-tuned model preferred in 78 of 100 comparisons
Length-Controlled Win Rate	**68.97 %**	Adjusted score removing response-length bias

Table 5. AlpacaEval Results

Qualitative Examples

Instruction	Base Model Output (excerpt)	Fine-Tuned Output (excerpt)
"How do I wrap a present neatly?"	Repeats "fold in half" with little clarity.	Provides structured steps – measure, fold, tape, add ribbon.
"What are the best exercises for beginners?"	Lists exercise names only.	Explains each exercise and its purpose (e.g. squats → leg strength).
"Why do cats rub against your legs?"	Generic "for attention" answer.	Correctly explains scent-marking and social bonding.
"Explain how photosynthesis works."	One-line definition without detail.	Clear description of reactants, products, and light energy process.

Table 6. AlpacaEval Qualitative Comparisons

The fine-tuned model consistently generated **more coherent, context-aware, and instructive** responses compared to the base model.

Performance Visualisation

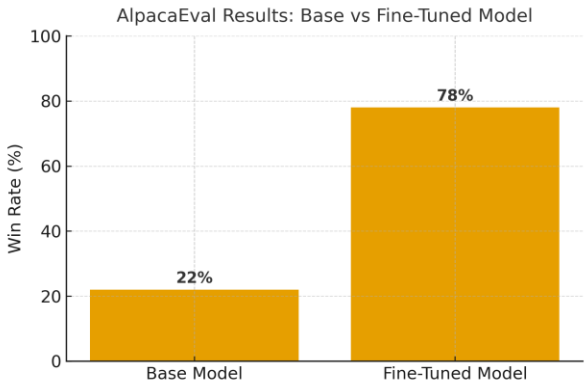


Image 2. AlpacaEval Win Rate Comparison between models

The chart shows that the fine-tuned model achieved a **78 % win rate**, outperforming the base model (22 %) in direct comparisons.

4.2 MT-Bench (Single-Turn)

To complement the single-turn AlpacaEval benchmark, the **MT-Bench** evaluation was conducted to further test the **base LLaMA-2-7B** and the **LoRA-fine-tuned** models. While MT-Bench is originally designed for multi-turn dialogue evaluation, this experiment used **only the first turn** of each prompt, creating a **single-turn test scenario** to maintain comparability with AlpacaEval.
All evaluations were performed on **Google Colab Pro** using an **NVIDIA A100 (High-RAM)** GPU.

Evaluation Setup

Component	Description
Dataset	MT-Bench (local JSONL copy, 80 questions)
Access Method	The dataset was downloaded and stored locally (question.jsonl) because the GitHub source frequently failed to load.
Base Model	meta-llama/Llama-2-7b-hf
Fine-Tuned Model	LoRA-adapted LLaMA-2-7B trained on Dolly-15k
Judge Model	GPT-4o-mini
Evaluation Samples	80
Metrics	Raw Win Rate and Tie Count

Table 7. MT-Bench Eval Setup

Evaluation Method

1. **Response Generation** – Both models generated single-turn responses for 80 MT-Bench prompts.

2. **Pairwise Judging – GPT-4o-mini** compared the two responses and judged them based on correctness, reasoning, coherence, and clarity, choosing "A (fine-tuned)", "B (base)", or "Tie".

3. **Metric Computation** –
   - **Raw Win Rate:** % of cases where the fine-tuned model's response was

preferred.

- **Ties:** Responses judged equal in quality.

Results

Metric	Score	Description
Raw Win Rate	**46.25 %**	Fine-tuned model preferred in 37 of 80 comparisons
Ties	**0**	No responses scored equally

Table 8. MT-Bench Eval Results

Analysis: Why MT-Bench Performance Was Lower

Although this MT-Bench evaluation was **single-turn only**, the fine-tuned model achieved a much lower **46.25 % win rate** compared to **78 %** on AlpacaEval. This decline stems from several technical and stylistic factors rather than multi-turn dialogue limitations.

1. **Higher Question Complexity**
   MT-Bench includes reasoning-heavy, domain-specific prompts (e.g., coding, STEM, philosophy), which are far more challenging than the concise factual questions in Dolly-15k.
   The fine-tuned model was never trained on such topics, leading to poorer generalisation.

2. **Formatting Bias from Dataset Preparation**
   During preprocessing, every sample followed a strict template. This rigid instruction format reinforces short, direct outputs. In MT-Bench, however, GPT-4 judges often favour **longer, more elaborative** and conversational answers—something the base model naturally produces.

3. **Over-Specialisation from LoRA Fine-Tuning**
   LoRA efficiently improves precision on instruction tasks but can narrow stylistic diversity. As a result, the fine-tuned model tends to generate **technically correct but terse and literal** responses, which appear less impressive in qualitative comparisons.

4. **Evaluation Bias toward Length and Fluency**
   Even in single-turn mode, GPT-4 judges tend to reward outputs with richer narrative flow and longer explanations. The fine-tuned model's concise structure, though clear, is often scored lower.

In short, the model's instruction-centric fine-tuning improved clarity but **reduced elaboration and stylistic flexibility**, explaining its lower MT-Bench score.

Qualitative Examples

Instruction	Base Model Output (excerpt)	Fine-Tuned Output (excerpt)
"Draft a debate between two scientists on climate change."	Balanced multi-paragraph discussion.	Short factual summary; lacks opposing voices.
"Give advice to a student who failed an exam."	Empathetic and motivational.	Direct, factual, but emotionally flat.
"Explain why space travel is important."	Multi-angle argument covering science and philosophy.	Focuses narrowly on technology.
"Pretend to be a tour guide in Paris."	Conversational tone with descriptive imagery.	Factual list of locations; no personality.

Table 9. MT-Bench Qualitative Examples

These examples show that the fine-tuned model produces **clear yet minimal** responses—strong for structured instructions, but weak for creative or open-ended tasks.

Conclusion

This project successfully fine-tuned **LLaMA-2-7B** using **Parameter-Efficient Fine-Tuning (LoRA)** on the **Databricks Dolly-15K** instruction dataset.
By cleaning, re-formatting, and splitting the dataset locally, and running three hours of training on a Colab A100 High-RAM GPU, the model demonstrated clear loss convergence (**training = 1.11**, **validation = 1.32**) while remaining memory-efficient.

Quantitative evaluation confirmed substantial improvement in instruction-following capability.
On **AlpacaEval 2**, the fine-tuned model achieved a **78 % win rate** over the base model, producing more coherent, contextually relevant, and human-aligned responses.
In contrast, **MT-Bench** yielded a lower **46 % win rate**, reflecting weaker multi-turn reasoning and long-context coherence—an expected limitation because **Dolly-15K** mainly contains single-turn instructions rather than sustained dialogues.
This illustrates how dataset style and prompt structure directly influence downstream benchmark performance.

Qualitative inspection further showed that fine-tuning improved factual accuracy, politeness, and formatting consistency while slightly reducing creativity and abstract reasoning.

In summary, this experiment demonstrates that **LoRA-based fine-tuning is an effective and resource-efficient strategy** for adapting large language models to task-specific behaviour.

Future improvements could include fine-tuning on **multi-turn conversational datasets** (e.g., ShareGPT, OpenAssistant) and exploring **RLHF or preference optimization** to enhance dialogue depth and contextual persistence.
Such extensions would likely boost MT-Bench and reasoning-oriented performance while preserving the strong instruction-following gains achieved here.

References

1. **Meta AI.** *LLaMA 2: Open Foundation and Fine-Tuned Chat Models.* Meta Platforms Inc., 2023. Available at https://ai.meta.com/llama/

2. **Hugging Face Transformers Library.** *Transformers Documentation and API.* GitHub repository, 2023. https://github.com/huggingface/transformers

3. **Hugging Face PEFT (LoRA).** *Parameter-Efficient Fine-Tuning Framework.* GitHub repository, 2023. https://github.com/huggingface/peft

4. **Databricks.** *Databricks Dolly 15K Dataset.* Hugging Face Datasets, 2023. https://huggingface.co/datasets/databricks/databricks-dolly-15k

5. **Tatsu Lab.** *AlpacaEval 2: Automatic Evaluation for Instruction-Following Models.* GitHub, 2024. https://github.com/tatsu-lab/alpaca_eval

6. **LMSYS Org.** *FastChat / MT-Bench: Open Benchmark for Chatbot Dialogue Evaluation.* GitHub, 2023. https://github.com/lm-sys/FastChat

7. **OpenAI.** *GPT-4o and GPT-4o-mini API Models Documentation.* https://platform.openai.com/docs